

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for the ensuring data security. (BL3, BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

APPLICATION- BASED QUESTIONS

Code of Conduct:

*I N.Siva venkata Trivikram (192525063). (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



APPLICATION- BASED QUESTIONS

1. Scenario: Event Communication Security
A student organization is preparing to share confidential event details with its core members using a simple classical encryption method. During testing, a message “MEET AT NOON” is transformed into an encoded version using a fixed shift pattern.

- Reconstruct how the encrypted message might appear if a shift of 4 positions is applied.
- Explain how an authorized member, aware of the encoding rule, would retrieve the original message.

2. Scenario: Intercepted Suspicious Message
A cybersecurity intern intercepts the coded message: “GSRH RH Z HVXIVG” during network monitoring. The structure of the text suggests a simple substitution mechanism rather than complex encryption.

- Deduce the most likely original message.
- Justify your answer by analyzing repeating patterns, letter structures, or known cipher characteristics.

3. Scenario: Secure Messaging System
A company deploys a keyword-based encryption technique for internal communication. An employee sends the word “HELLO” using the keyword “KEY” to ensure confidentiality.

- Demonstrate how the encryption process transforms the message step-by-step.
- Explain how the repeating keyword influences each character of the ciphertext.

4. Scenario: Suspicious Digital Asset Investigation
During a digital forensic audit, investigators suspect that a company logo file may contain hidden confidential information embedded within it. The file appears normal upon casual inspection.

- Analyze how investigators could detect the presence of hidden data within the image.
- Recommend two techniques that could strengthen the security of such hidden communications against detection or extraction.

5. Scenario: Legacy Encryption in Corporate Communication
A legacy system in a company still uses a transposition-based encryption method with three levels of arrangement. A received message appears as: “WECRLTEERDSOEFEAOCAIVDEN”.

- a. Determine the most probable original message by reversing the encryption process.
- b. Critically evaluate why such a method is vulnerable in modern communication systems.
- c. Propose one practical enhancement to improve the confidentiality of messages in this scenario.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use	20	Effective and advanced use of tools/technologies	Appropriate use of tools	Limited or inconsistent use of tools	Incorrect or minimal

of Too ls		s with proper configuration	with correct execution		use of tools
-----------------	--	--------------------------------	------------------------------	--	-----------------

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonab le interpreta tio n	Basic analysis with limited insights	Poor or incorec t analysis of results
Document at ion	10	Clear, well- structured documentati on with diagrams, code, and results	Organized documenta ti on with necessary details	Basic documenta tio n with missing details	Poor or incomplete documentat io n

SIMATS ENGINEERING

Question 1: Event Communication Security

Scenario

A student organization shares confidential event details with its core members using a simple classical encryption method. The message “MEET AT NOON” is encoded using a fixed shift pattern.

- (a) Reconstruct how the encrypted message appears if a shift of 4 positions is applied.
- (b) Explain how an authorized member, aware of the encoding rule, retrieves the original message.

1. Understanding of Concepts

A fixed shift of letters is the **Caesar cipher**, the oldest and simplest **monoalphabetic substitution** cipher. Every plaintext letter is replaced by the letter a fixed number of positions further along the alphabet. It is a **symmetric** cipher: the same secret (the shift value) is used to encrypt and decrypt.

Term	Meaning in this question
Plaintext (P)	MEET AT NOON – the readable message
Ciphertext (C)	The encoded message sent to members
Key (k)	Shift value = 4 (the shared secret “encoding rule”)
Alphabet size	26 letters, numbered A = 0, B = 1, ... Z = 25
Encryption rule	$C = (P + k) \bmod 26$
Decryption rule	$P = (C - k) \bmod 26$ (add 26 if the result is negative)

The “mod 26” operation makes the alphabet circular, so after Z the count wraps back to A (e.g., $W + 4 \rightarrow A$).

2. Design of the Solution (Method)

- Convert each letter to its numeric position ($A = 0 \dots Z = 25$).
- Add the key 4 and take the result modulo 26.
- Convert the new number back to a letter; spaces are left unchanged.
- Decryption applies the inverse operation (subtract 4) in the same manner.

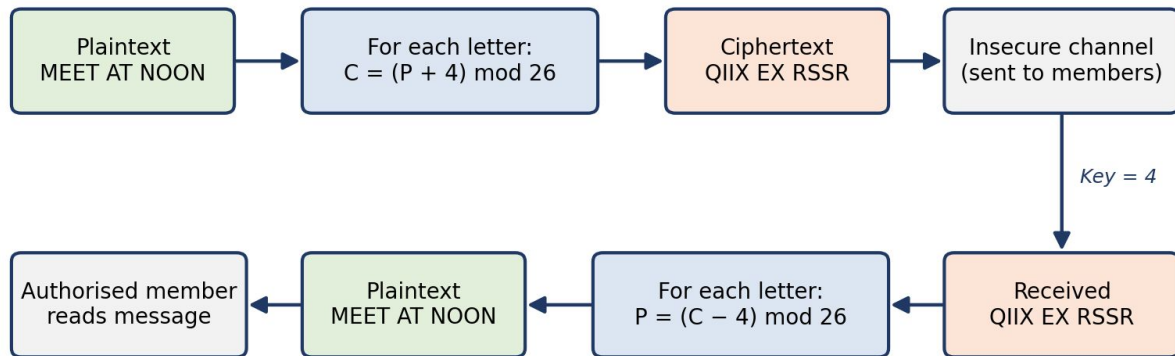
Caesar Cipher (Shift = 4): Encryption and Decryption Flow

Figure 1: Flowchart of Caesar encryption (sender) and decryption (receiver), shift = 4

Plain alphabet → Cipher alphabet (shift 4)

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
Cipher	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D

Highlighted = letters used in "MEET AT NOON"

Figure 2: Plain alphabet aligned with the shift-4 cipher alphabet

3. Solution**(a) Encryption with a shift of 4**

Plain letter	Position (P)	P + 4	(P + 4) mod 26	Cipher letter
M	12	16	16	Q
E	4	8	8	I
E	4	8	8	I
T	19	23	23	X
A	0	4	4	E
T	19	23	23	X
N	13	17	17	R
O	14	18	18	S
O	14	18	18	S
N	13	17	17	R

Result: MEET AT NOON → QIIX EX RSSR (M → Q, E → I, E → I, T → X | A → E, T → X | N → R, O → S, O → S, N → R)

(b) Decryption by an authorized member

The member knows the rule (Caesar cipher, shift = 4). Decryption moves every letter 4 positions backwards:

- Step 1 – Identify the algorithm and key from the shared rule: Caesar, $k = 4$.
- Step 2 – Convert each ciphertext letter to its position number.
- Step 3 – Compute $P = (C - 4) \bmod 26$. If the value is negative, add 26 (e.g., $A \rightarrow 0 - 4 = -4 \rightarrow 22 \rightarrow W$).
- Step 4 – Convert numbers back to letters and keep the word spaces.

Cipher letter	Position (C)	$C - 4$	$(C - 4) \bmod 26$	Plain letter
Q	16	12		M
I	8	4		E
I	8	4		E
X	23	19		T
E	4	0		A
X	23	19		T
R	17	13		N
S	18	14		O
S	18	14		O
R	17	13		N

Result: QIIX EX RSSR $\rightarrow$ **MEET AT NOON** (equivalently, encrypting with the complementary key $26 - 4 = 22$ gives the same answer).

4. Tools Used

- **Manual method:** modular-arithmetic tables and the shifted alphabet (Figure 2).
- **Python 3 program:** implements encryption, decryption and a brute-force check to validate the manual result (code and output below).
- **Optional cross-check:** any Caesar tool such as CrypTool or an online Caesar decoder, with shift 4.

Python implementation (Question 1 part)

```
ALPHA = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
def caesar(text, shift):
    result = ""
    for ch in text.upper():
        if ch in ALPHA:
            result += ALPHA[(ALPHA.index(ch) + shift) % 26]
        else:
            result += ch          # keep spaces unchanged
    return result
```

```
# ----- Question 1 -----
```

```
plain = "MEET AT NOON"
cipher = caesar(plain, 4)
print("Q1 Plaintext :", plain)
print("Q1 Ciphertext :", cipher)
print("Q1 Decrypted :", caesar(cipher, -4))
print("Q1 Brute force on ciphertext (all 25 keys):")
for s in range(1, 26):
    print(f"  key {s:2d} -> {caesar(cipher, -s)}")
```

Program output (Question 1)

```

Q1 Plaintext : MEET AT NOON
Q1 Ciphertext : QIIX EX RSSR
Q1 Decrypted : MEET AT NOON
Brute-force excerpt:
key 1 -> PHHW DW QRRQ
key 3 -> NFFU BU OPPO
key 4 -> MEET AT NOON
key 5 -> LDDS ZS MNNM
key 22 -> UMMB IB VWWV

```

5. Analysis of Results

- **Correctness:** decrypting QIIX EX RSSR with $k = 4$ returns exactly MEET AT NOON, so the cipher round trip is lossless; the manual tables and the program agree.
- **Pattern preservation:** the double letters EE $\rightarrow$ II and OO $\rightarrow$ SS, the repeated T $\rightarrow$ X and N $\rightarrow$ R, and the word lengths (4-2-4) all survive encryption. An attacker therefore learns the message structure even without the key.
- **Key space:** only 25 usable shifts exist. The brute-force output shows that trying all of them takes a fraction of a second, and only key 4 yields meaningful English.
- **Frequency weakness:** because each letter always maps to the same cipher letter, the letter-frequency profile of the language is only shifted, not hidden.
- **Conclusion:** a shift-4 Caesar cipher is suitable for demonstrating CO1 concepts, but not for protecting truly confidential data; stronger polyalphabetic (see Question 3) or modern ciphers (AES) are required.

Question 3: Secure Messaging System

Scenario

A company deploys a keyword-based encryption technique for internal communication. An employee sends the word “HELLO” using the keyword “KEY”.

- (a) Demonstrate how the encryption process transforms the message step by step.
- (b) Explain how the repeating keyword influences each character of the ciphertext.

1. Understanding of Concepts

A keyword-based cipher with a repeating key is the **Vigenère cipher**, a **polyalphabetic substitution** cipher. Instead of one fixed shift (Caesar), each letter of the keyword defines a different shift. The keyword is repeated until it covers the entire message, so successive plaintext letters are encrypted with different Caesar shifts. It is symmetric: the same keyword is used to decrypt.

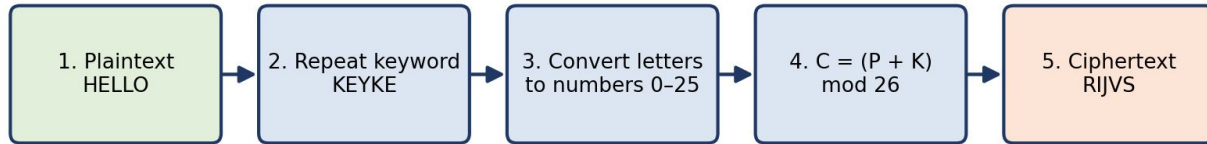
Term	Meaning in this question
Plaintext (P)	HELLO
Keyword (K)	KEY – the shared secret
Repeated key stream	KEYKE (KEY repeated and cut to the message length of 5)
Letter numbering	A = 0, B = 1, ... Z = 25
Encryption rule	$C_i = (P_i + K_i) \bmod 26$
Decryption rule	$P_i = (C_i - K_i + 26) \bmod 26$

2. Design of the Solution (Method)

- Write the plaintext and repeat the keyword beneath it, letter for letter.
- Convert both rows to numbers (A = 0 ... Z = 25).

- Add each plaintext number to the matching key number and reduce modulo 26.
- Convert the results back to letters to obtain the ciphertext.

Vigenère Encryption Process (Plaintext HELLO, Keyword KEY)



Decryption reverses step 4: $P = (C - K + 26) \bmod 26$

Figure 3: Flowchart of the Vigenère encryption process

3. Solution

(a) Step-by-step encryption

Step 1 – Align plaintext and repeated keyword:

Position	1	2	3	4	5
Plaintext	H	E	L	L	O
Keyword (repeated)	K	E	Y	K	E

Step 2 – Convert letters to numbers:

Position	1	2	3	4	5
Plaintext number (P)	7	4	11	11	14
Key number (K)	10	4	24	10	4

Step 3 – Add and reduce modulo 26:

Position	P	K	P + K	(P + K) mod 26	Cipher letter
1	7 (H)	10 (K)	17	17	R
2	4 (E)	4 (E)	8	8	I
3	11 (L)	24 (Y)	35	35 – 26 = 9	J
4	11 (L)	10 (K)	21	21	V
5	14 (O)	4 (E)	18	18	S

Step 4 – Combine the letters:

HELLO + KEY → RIJVS (H+K → R, E+E → I, L+Y → J, L+K → V, O+E → S)

The Vigenère square gives the same result: the plaintext letter selects the column, the key letter selects the row, and the intersection is the cipher letter (highlighted cells in Figure 4).

Key ↓ / Plain → **Vigenère Square (Tabula Recta) - cells used for HELLO + KEY**

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

H+K→R E+E→I L+Y→J L+K→V O+E→S

Figure 4: Vigenère square with the five cells used to encrypt HELLO with key KEY

Verification by decryption

Position	C	K	C – K	+26 if negative	Plain letter
1	17 (R)	10	7	7	H
2	8 (I)	4	4	4	E
3	9 (J)	24	-15	11	L
4	21 (V)	10	11	11	L
5	18 (S)	4	14	14	O

(b) Influence of the repeating keyword on each ciphertext character

Each keyword letter acts as a Caesar shift equal to its alphabet position (K = 10, E = 4, Y = 24). Because KEY has length 3, the shifts repeat in a cycle of three:

Position	Plain	Key letter	Shift applied	Cipher	Effect
1	H	K	+10	R	Shifted by cycle position 1
2	E	E	+4	I	Shifted by cycle position 2
3	L	Y	+24	J	Shifted by cycle position 3
4	L	K	+10	V	Cycle restarts (same shift as position 1)
5	O	E	+4	S	Same shift as position 2

- **Same letter, different outputs:** the two L's in HELLO become J and V, because they meet different key letters (Y and K). In a Caesar cipher they would both give the same letter. This hides double letters and flattens letter frequencies.

- **Different letters, same shift:** H (position 1) and L (position 4) are both shifted by $K = 10$, showing the period of 3 introduced by the keyword length.
- **Each letter depends on its position:** the cipher letter depends on both the plaintext letter and its place in the key cycle, so the cipher has 3 independent Caesar alphabets ($26^3 = 17,576$ possible 3-letter keys).
- **Longer keywords give more security:** the shift pattern repeats every $\text{len}(\text{key})$ letters; a key as long as the message and used once would be the one-time pad (unbreakable).

4. Tools Used

- **Manual method:** tabular modular arithmetic and the Vigenère square (Figure 4).
- **Python 3 program:** encrypts, shows each step, decrypts, and tests a wrong keyword.
- **Optional cross-check:** CrypTool or any online Vigenère calculator with keyword KEY.

Python implementation (Question 3 part)

```
def vigenere(text, key, decrypt=False):
    result, k = "", 0
    for ch in text.upper():
        if ch in ALPHA:
            shift = ALPHA.index(key[k % len(key)].upper())
            if decrypt:
                shift = -shift
            result += ALPHA[(ALPHA.index(ch) + shift) % 26]
            k += 1
        else:
            result += ch
    return result

# ----- Question 3 -----
msg, key = "HELLO", "KEY"
rep_key = (key * 5)[:len(msg)]
print("\nQ3 Plaintext :", msg)
print("Q3 Keyword   :", key, "-> repeated:", rep_key)
for p, k in zip(msg, rep_key):
    P, K = ALPHA.index(p), ALPHA.index(k)
    print(f" {p}{P:2d} + {k}{K:2d} = {P+K:2d} mod 26 = {(P+K)%26:2d} -> {ALPHA[(P+K)%26]}")
enc = vigenere(msg, key)
print("Q3 Ciphertext :", enc)
print("Q3 Decrypted  :", vigenere(enc, key, decrypt=True))
print("Q3 Wrong key 'ABC' gives:", vigenere(enc, "ABC", decrypt=True))
```

Program output (Question 3)

```
Q3 Plaintext : HELLO
Q3 Keyword   : KEY -> repeated: KEYKE
H( 7) + K(10) = 17 mod 26 = 17 -> R
E( 4) + E( 4) =  8 mod 26 =  8 -> I
L(11) + Y(24) = 35 mod 26 =  9 -> J
L(11) + K(10) = 21 mod 26 = 21 -> V
O(14) + E( 4) = 18 mod 26 = 18 -> S
Q3 Ciphertext : RIJVS
Q3 Decrypted  : HELLO
Q3 Wrong key 'ABC' gives: RHHVR
```

5. Analysis of Results

- **Correctness:** manual tables, the Vigenère square and the program all give RIJVS; decryption returns HELLO.
- **Wrong key test:** decrypting RIJVS with the keyword ABC gives the gibberish “RHHVR” rather than HELLO, confirming that the exact keyword is required.
- **Improvement over Caesar (Question 1):** repeated plaintext letters no longer produce repeated cipher letters, and the 25-key brute force is replaced by a keyword search space of 26^n for an n -letter keyword.

- **Remaining weakness:** the key repeats, so with longer messages the Kasiski examination and the Index of Coincidence can reveal the key length, after which each position group is broken like a Caesar cipher by frequency analysis. Long, random, non-dictionary keywords reduce this risk.
 - **Conclusion:** the Vigenère cipher offers noticeably stronger confidentiality than a single shift cipher, but modern symmetric ciphers (AES) are still needed for real corporate communication.
-